

第5章 散列

建议学时:4-5 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言,DS&A 零基础(已读完第 1-4 章)

🎯 本章学习目标

- 理解散列为什么能把查找从 $O(\log n)$ 压到平均 $O(1)$:散列函数把键映射为数组下标,理想情况下一步直达。
- 掌握整数取模与字符串 37 倍多项式两种散列函数,能说出"为什么表大小必须是素数、为什么要最后才取模"。
- 掌握分离链接法与开放定址法(线性探测、平方探测、双散列)的实现,理解装填因子 λ 与惰性删除。
- 掌握再散列的动机与实现,说出摊还代价 $O(1)$;了解可扩散列的基本思想。
- 学会用 `unordered_map` / `unordered_set` 解决问题,能给自定义类型写哈希与判等。
- 能独立实现词频统计程序(手写 + 标准库两条路线),用实验数据理解装填因子的影响。

💡 从 C 到 C++:本章主题如何迁移

数组按下标访问是 $O(1)$ 的,前提是下标是 0、1、2、... 这样的连续整数。可现实里的键往往不连续:学号是 2023001、2023050,订单号是一串字母数字。散列的想法很简单:写一个函数把"任意的键"翻译成"一个合理的下标",然后直接进数组。这个函数就叫 **散列函数**。

C 语言里你要自己写这个函数,并处理两个麻烦:负数取模会得到负下标(必须 `if (h < 0) h += M`),字符串散列时 `int` 容易溢出。到了 C++,麻烦基本消失:`unsigned long long` 累加几乎不可能溢出,标准库更是直接做好了"散列容器"。

本章主线是"先手写,再用库":§5.2 用 `vector` 和 `list` 拼出分离链接散列表——桶就是链表,是 C 语言"结构体 + 指针"的现代化版本;§5.7 的单词统计证明手写版与 `unordered_map` 输出完全一致。散列以空间换时间、以顺序换速度:查找快但遍历无序,与第 4 章二叉查找树正好相反。

📦 前置知识自查

数学前置:模运算(取余),重点是负数取模在 C/C++ 中是"截断"语义: `a % b` 结果可正可负,散列里必须处理。另一条是素数:只能被 1 和自身整除的正整数。

C 语言前置:指针遍历字符串、`struct` 与链表节点、`malloc/free`。本章的 C 对照会反复用到这三样。

依赖的前面章节:第 3 章链表与 `vector`——分离链接法的桶就是链表;再散列"翻倍扩容再重插"与 `vector` 扩容同构。

🚀 本章学习路线

1. **第一阶段:**读 §5.1-§5.2,吃透散列函数与分离链接法;手算教材 5.2 节小例子,对照示例 5-3 抄一遍代码。
2. **第二阶段:**读 §5.3-§5.5,理解三个探测方案的"聚集"问题与平方探测定理;读 §5.6 时敲一遍 `unordered_map` 示例。
3. **第三阶段:**完成章末实验,手写表与 `unordered_map` 双路线统计词频,看 λ 对性能的影响。

本章知识导图

- §5.1 基本想法与散列函数
 - 5.1.1 基本想法:查找的第三件武器
 - 5.1.2 整数键:取模散列与素数表大小
 - 5.1.3 字符串键:37 倍多项式散列
 - 5.1.4 装填因子 λ
- §5.2 分离链接法
 - 5.2.1 桶就是一条链表
 - 5.2.2 完整实现:查找、插入与删除
 - 5.2.3 复杂度:平均 $O(1)$ 从哪来
- §5.3 开放定址法
 - 5.3.1 线性探测与一次聚集
 - 5.3.2 平方探测
 - 5.3.3 双散列
 - 5.3.4 三种方案对比与惰性删除
- §5.4 再散列
 - 5.4.1 什么时候需要再散列
 - 5.4.2 实现:翻倍、取素数、重插
 - 5.4.3 摊还代价
- §5.5 可扩散列(选读)
- §5.6 标准库中的散列结构
 - 5.6.1 `unordered_map` 与 `unordered_set`
 - 5.6.2 自定义类型的哈希
 - 5.6.3 选型:`unordered_map` 还是 `map`
- §5.7 手写与标准库并排:单词统计

§5.1 基本想法与散列函数

5.1.1 基本想法:查找的第三件武器

你学过两种查找:顺序扫描(链表, $O(n)$)和二叉查找树(平均 $O(\log n)$)。散列是第三条路:把每个键换算成数组下标,查找就是一次数组访问,平均 $O(1)$ 。

表大小为 M ,散列函数 `hash(x)` 把键 x 映射到 0 到 $M-1$ 。插入: `table[hash(x)] = x`;查找:查 `table[hash(x)]` 里有没有 x 。难点在于散列函数"多对一":不同键可能映射到同一下标,这叫 **冲突**。散列的学问 = 好的散列函数(冲突少)+ 好的冲突解决策略。

记住 **散列三要素**:散列函数(键 $\rightarrow$ 下标)+ 冲突解决策略(§5.2 分离链接 / §5.3 开放定址)+ 表大小 M (尽量取素数)。任一环做差,性能都会从"平均 $O(1)$ "掉回"最坏 $O(n)$ "。

5.1.2 整数键:取模散列与素数表大小

键是整数时最常用的散列函数是取模: $\text{hash}(x) = x \% M$, 开销只有一次除法。关键是表大小 M 的选择: 若 M 是合数而键恰好都带某个公因子, 就会全挤进少数几个桶。比如 $M = 10$, 键 42、32、52、72 的个位全是 2, $x \% 10$ 全部等于 2——四键同桶, 九桶全空; 取素数 $M = 11$ 则余数 9、10、8、6, 四散而开。键的分布未知时, 素数表是唯一稳妥的选择。

⚠ 误区: 负数取模会得到负数下标

C/C++ 的 $\%$ 是截断除法: $-5 \% 7$ 等于 -5 而不是 2 。直接拿负值当数组下标会越界(未定义行为), 散列函数内部必须保证结果非负:

```
// 负数取模示例:C/C++ 的 % 是截断除法, 结果可正可负
int h = x % M;           // x = -5, M = 7 时 h = -5, 不是 2!
cout << h;              // 会打印 -5, 负数下标会越界
if (h < 0)
    h += M;              // 把负数修正到 [0, M-1]
return h;                // 现在可以放心当数组下标用了
// 更省心的惯用写法: unsigned(x) % M 直接消掉负号
// 两种写法殊途同归: 散列结果必须落在 [0, M-1]
```

C 的写法	C++ 的写法
int h = x % M; 负数得到负下标, 要手动 if (h < 0) h += M;	同样要处理; unsigned(x) % M 一行消掉负号, 或同样写 if 修正
int 累加散列容易溢出, 结果随机甚至变负	unsigned long long 累加, 64 位几乎不会溢出(见 5.1.3)

5.1.3 字符串键: 37 倍多项式散列

字符串是最常见的键(单词、文件名、URL)。教材采用 37 倍多项式法, 把字符串按霍纳法则逐位乘加, 每步 $h = h * 37 + c$ 。先给 C 回顾版, 再给 C++ 写法:

示例 5-1 C 回顾: 字符串散列 (37 倍多项式, 霍纳法则)

```
/* C 回顾: 字符串散列 */
int hashStrC(char* s, int M) {
    unsigned int h = 0;           /* unsigned 累加, 防止溢出成负数 */
    while (*s != '\0') {
        h = h * 37 + (unsigned char)*s; /* 每步乘 37 再加当前字符 */
        ++s;
    }
    return (int)(h % M);          /* 最后才取模, 结果落在 [0, M-1] */
}
```

示例 5-2 C++ 版: 同样的逻辑, 更短的代码

```
// C++ 版: 用 std::string 与范围 for, 逻辑与 C 完全对应
int hashStr(const string& s, int M) {
    unsigned long long h = 0;      // 64 位累加, 几乎不可能溢出
    for (char c : s)               // 逐个字符取码, 累乘累加
        h = h * 37 + (unsigned char)c;
    return int(h % M);             // 最后取一次模, 结果在 [0, M-1]
}
// 用法示例: hashStr("hello", 101) 返回 0..100 之间的整数
```

两句代码除类型与遍历方式外一字不差;"最后取一次模"几个字不是风格,是正确性——见下面的误区。

 **误区:逐字符取模会悄悄毁掉散列**

有人图省事,把取模写进循环:

```
// 错误示范:把取模写进循环,乘法被取模"吃掉"
for (char c : s)
    h = (h * 37 + (unsigned char)c) % M;    // 错误!
// 当 M = 37 时, (h*37+c)%37 = c%37,
// 散列退化成只看最后一个字符;
// 当 M = 36 时, 37 ≡ 1 (mod 36),
// "abc" 与 "cba" 算出同一个散列值,
// 字符串变成了"字符和"。
```

两种退化同源:取模发生在乘法之前,丢掉了进位。正确写法永远是先累加、最后取一次模(示例 5-2)。

5.1.4 装填因子 λ

装填因子 $\lambda = \text{元素个数 } N / \text{表大小 } M$,衡量表"满"的程度: λ 越大冲突越多,查找越慢; λ 越小空位越多,浪费越大。分离链接的经验阈值 $\lambda \approx 0.75$ (再大就该扩容,见 §5.4);开放定址更苛刻,平方探测要求 $\lambda < 1/2$ 。

 **记号说明: λ 的两种记法**

正文用希腊字母 λ ;代码里命名为 `loadFactor()` (或 `load_factor`),实现为 N / M 的浮点除法,两者是同一个量。

§5.2 分离链接法

5.2.1 桶就是一条链表

分离链接法(separate chaining)的思路最直观:每个下标放一条链表(教材图 5-1)。插入:算桶号挂到链表;查找:算桶号,链内线性扫描;删除:算桶号,链中摘除。 M 决定链表条数, $\lambda = N/M$ 就是平均链长。

C 的写法	C++ 的写法
<pre>struct Node { char* key; Node* next; }; 手写链表头插法,自己 malloc/free 每个节点</pre>	<code>vector<list<string>></code> :外层 vector 是桶数组,内层 list 是桶,内存自动管理
散列后用 <code>strcmp</code> 逐个比较	范围 for + <code>==</code> 直接比较 string

5.2.2 完整实现:查找、插入与删除

下面是完整 C++ 实现。`nextPrime(M)` 返回不小于 M 的最小素数(实现见章末实验),构造函数用它定素数表大小。

示例 5-3 分离链接散列表:`find / insert / remove`

```
// 分离链接散列表:桶数组 vector<list<string>>,一个桶就是一条链表
class HashTable {
    vector<list<string>> buckets_; // 桶数组:每个元素是一条链表
    int size_ = 0; // 表中元素总个数
public:
    explicit HashTable(int M)
        : buckets_(nextPrime(M)) {} // 表大小必须是素数($5.1.2)

    int hash(const string& s) const { // 37 倍多项式散列
        unsigned long long h = 0;
        for (char c : s)
            h = h * 37 + (unsigned char)c;
        return int(h % buckets_.size());
    }

    bool find(const string& s) const { // 查找:先定桶,再在链内找
        for (const string& x : buckets_[hash(s)])
            if (x == s) return true;
        return false;
    }

    void insert(const string& s) { // 插入:已存在则跳过
        if (find(s)) return;
        buckets_[hash(s)].push_back(s); // 挂到对应桶的链表尾
        ++size_;
    }

    void remove(const string& s) { // 删除:定位桶后链内删除
        list<string>& b = buckets_[hash(s)];
        for (auto it = b.begin(); it != b.end(); ++it)
            if (*it == s) { b.erase(it); --size_; return; }
    }

    int size() const { return size_; }
    double loadFactor() const { // 装填因子  $\lambda$  = 元素数/表大小
        return double(size_) / buckets_.size();
    }
};
```

三个操作同一套路:先 `hash(s)` 定桶,再只动那一条链——散列把"全表扫描"降为"单桶扫描",单桶平均只有 λ 个元素。
`insert` 先查重再 `push_back` (散列两次;教材图 5-6 的版本合并为一次);`remove` 定位后 `erase(it)` 摘除。

⚠ 误区:erase 之后必须立刻返回,否则迭代器失效

`list::erase(it)` 会令被删位置的迭代器失效。上面 `remove` 里 `erase(it)` 后立刻 `return` 就没事;若删除后继续用 `it`(如 `++it`),就是未定义行为——C++ 新手最常见的散列表 bug。vector 的 `erase` 更激进:被删位置之后的迭代器全部失效。

5.2.3 复杂度:平均 $O(1)$ 从哪来

一次 `find` 的代价 = 散列 $O(\text{键长 } L)$ + 单桶扫描 $O(\text{平均链长}) = O(1 + \lambda)$ 。 λ 控制在 0.75 左右时,常数 $1 + \lambda$ 很小,于是散列表查找 **平均 $O(1)$** ;插入、删除同理。

牢记散列表的复杂度画像:查找/插入/删除平均 $O(1)$ 、最坏 $O(n)$ 。最坏情况是所有键散列到同一桶,链表退化成一条长链——散列函数均匀且表大小是素数时几乎不会发生,但你必须能说出它为什么是 $O(n)$ 。代价是牺牲顺序性:无法按键有序遍历、没有前驱后继;要顺序时回到第 4 章的二叉查找树。

§5.3 开放定址法

分离链接需要额外指针,浪费空间。开放定址法(open addressing)让所有元素都住进表本身:冲突时按某种规则在表中找下一个空位。规则不同,衍生出三种方案。

5.3.1 线性探测与一次聚集

线性探测:冲突时逐个向后试 $\text{pos} = (\text{pos} + 1) \% M$,直到空位或绕回。它有个著名毛病——一次聚集:连续被占的单元越滚越长,新键插进去又让它更长,形成“雪球”。

示例 5-4 线性探测插入

```
// 开放定址:线性探测插入(空单元用 EMPTY 标记)
int pos = hashStr(s, M);
while (table[pos] != EMPTY && table[pos] != s)
    pos = (pos + 1) % M;    // 冲突就向后挪一格,越界绕回 0
if (table[pos] == EMPTY)
    table[pos] = s;        // 找到空位,插入
else
    ;                      // 键已存在,不做任何事
```

手算一遍: $M = 7$,按序插入 7、15、8、1($h(x) = x \bmod 7$): $7 \rightarrow 0$; $15 \rightarrow 1$; $8 \rightarrow 1$ 冲突试 2,空,落位; $1 \rightarrow 1$ 冲突试 2 仍冲突,试 3,落位。最终下标 0-3 依次是 7、15、8、1:一次聚集——三键挤成一团,后面的键要从 1 一路试到 4。

5.3.2 平方探测

平方探测把步长改成平方:第 i 次冲突后尝试 $\text{pos} + i^2$,探测序列从一开始就跳开,避免连续占用。

示例 5-5 平方探测插入

```
// 平方探测:第 i 次冲突后尝试 pos + i^2
int pos = hashStr(s, M);
int i = 0;
while (table[pos] != EMPTY && table[pos] != s) {
    ++i;
    pos = (pos + i * i) % M;    // 探测序列:pos+1, pos+4, pos+9, ...
}
if (table[pos] == EMPTY)
    table[pos] = s;
// 定理(教材 5.4 节):表大小为素数且装填因子  $\lambda < 1/2$  时,
// 平方探测总能找到空单元,且最多探测一半的表长
```

教材定理:表大小是素数且 $\lambda < 1/2$ 时,平方探测总能在最多一半的表长内找到空单元(证明要点:前半段探测位置互不相同)。工程含义: λ 必须死守在半表以下,因此要更早再散列。它消灭了一次聚集,但留下“二次聚集”:同位置的键探测序列完全相同。

5.3.3 双散列

双散列连二次聚集也消灭:步长由第二个散列函数算出,每个键拥有专属探测节奏。常见选择:步长 = $M_2 - (h(x) \bmod M_2)$, M_2 取比 M 小的素数。

示例 5-6 双散列插入

```
// 双散列:冲突时以第二个散列函数的值做步长
int h1 = hashStr(s, M);           // 主散列:决定初始位置
int step = hashStr2(s, M);        // 第二散列:决定步长,注意不能为 0
int pos = h1;
while (table[pos] != EMPTY && table[pos] != s)
    pos = (pos + step) % M;        // 每次前进 step 格
if (table[pos] == EMPTY)
    table[pos] = s;
```

5.3.4 三种方案对比与惰性删除

方案	探测序列	代价
线性探测	顺次 +1	一次聚集, λ 高时退化严重;实现最简单
平方探测	+1, +4, +9, ...	无一次聚集,有二次聚集;需素数表且 $\lambda < 1/2$
双散列	步长由第二个散列决定	无聚集,但要多算一次散列函数

⚠ 误区:开放定址表不能物理删除

线性探测里删掉一个元素若直接置空,会把探测链拦腰截断:后续依赖它"借道"的元素从此再也找不到。教材做法是**惰性删除**:标记 deleted,查找跳过、插入复用、定期再散列清标记。注意 deleted 单元不算"已占用",装填因子计算要把它排除。

§5.4 再散列

5.4.1 什么时候需要再散列

装填因子涨过阈值后冲突率陡增,平均链长(或探测次数)失控。唯一的办法是**再散列**:表扩容(通常翻倍取素数),旧元素用新表大小重新散列。插入时检查 λ , 超阈值触发。工程阈值:分离链接约 0.75, 开放定址约 0.5。

5.4.2 实现:翻倍、取素数、重插

示例 5-7 再散列:给分离链接表扩容

```
// 再散列:装填因子超过阈值就把表翻倍并重新插入全部元素
void rehash() {
    if (loadFactor() < 0.75) return;           //  $\lambda$  未超标则不做
    int oldM = int(buckets_.size());
    vector<list<string>> oldBuckets = buckets_; // 先备份旧桶
    buckets_.assign(nextPrime(oldM * 2), {});   // 新表:翻倍后取素数
    size_ = 0;
    for (auto& chain : oldBuckets)              // 重新插入每个旧元素
        for (const string& x : chain)
            insert(x);                          // 会用新表大小重算散列
}
```

最后是"重新插入"而非"搬过去":桶号 = $\text{hash}(x) \bmod \text{新表大小}$,每个元素都得重算——"再散列"之名由此而来。

5.4.3 摊还代价

单次再散列 $O(n)$,但把账摊到 n 次插入上:每次扩容后空间翻倍,平均下来 每次插入摊还 $O(1)$ 。这与第 3 章 vector 扩容的摊还分析同构,区别是散列表翻倍后元素必须重插(不能拷贝)。

实现建议:再散列是"懒"的,只在需要时触发

教材每次插入后检查 λ ,超标才触发;也有按"插入次数达到 $n/2$ "预约的实现。关键是别等 λ 飙到 0.9 才扩容——那时聚集已形成,再散列也救不回。标准库 `unordered_map` 自动再散列,可用 `reserve()` 预分配桶数,避开插入高峰期扩容。

§5.5 可扩散列(选读)

5.5.1 散列与磁盘的冲突

前面所有方案都假设表在内存。数据量大到必须存磁盘时,一次磁盘 I/O 是内存访问的几十万倍,而散列函数均匀分布意味着每次查找都可能访问不同页,随机 I/O 会让性能崩溃。可扩散列(extendible hashing)专治"键太多,一页装不下"。

5.5.2 目录、页与局部分裂

可扩散列分两层:内存里的 目录 (directory,大小为 2^d 的数组)和一组磁盘 页 (page,一页容纳若干键)。查找:取散列值的前 d 位定位目录项,读目录项指向的页。插入时若页满就分裂:满页一分为二,目录规模翻倍(d 加一)。只有满的那一页被拆,其它页原封不动,这叫 局部分裂 ——表随数据增长逐页扩张,而非整体重建。

对照再散列:整表翻倍、全部重插(摊还 $O(1)$);可扩散列只分裂满页 + 目录翻倍,其它页不动,且只有一页 I/O。代价:目录占内存,可能有两页指向同一目录区段(浪费)。数据库与文件系统的索引设计正是这一思想的生产级版本。

§5.6 标准库中的散列结构

5.6.1 unordered_map 与 unordered_set

C++11 起标准库提供散列容器: `unordered_map<K, V>` (键值对)与 `unordered_set<K>` (键集合)。unordered 提醒你:遍历无顺序。底层就是本章讲的分离链接散列表(链表桶),平均 $O(1)$,自动再散列。

示例 5-8 unordered_map 基本用法

```
// 标准库:unordered_map(散列)与 map(红黑树)对单词计数
#include <unordered_map>
unordered_map<string, int> cnt;
for (const string& w : words)
    ++cnt[w];           // w 不存在时自动插入,次数从 0 加 1

// 遍历:按散列顺序,无任何顺序保证
for (auto& kv : cnt)
    cout << kv.first << ": " << kv.second << "\n";

// 三种查找姿势
auto it = cnt.find("hello");    // 返回迭代器;找不到返回 end()
if (it != cnt.end())
    cout << it->first << ": " << it->second << "\n";
if (cnt.count("hello"))        // 只想知道在不在:count() 返回 0 或 1
    cout << cnt.at("hello");    // at():找不到会抛 out_of_range 异常
```

最容易错的是 `operator[]` 的副作用: `cnt[w]` 在键不存在时会 **自动插入默认值**,只读查询请用 `find` / `count` / `at`,否则表里会冒出"幽灵键"。

5.6.2 自定义类型的哈希

`unordered` 容器要求键可判等(==)且可哈希。内置类型(int、string 等)标准库备好了散列;自定义类型要自己写一个函数对象作为第二个模板参数,这是教材推荐的做法:

示例 5-9 给自定义类型提供哈希与判等

```
// 自定义类型进散列表:标准库要求 可哈希 + 可判等
struct Student {
    string name;
    int id;
    bool operator==(const Student& o) const {    // 判等:成员全部相等
        return name == o.name && id == o.id;
    }
};

struct StudentHash {                            // 函数对象:散列函数
    size_t operator()(const Student& s) const {
        return hash<string>()(s.name) ^ hash<int>()(s.id);
    }
};

unordered_set<Student, StudentHash> roster;      // 用法:两个模板参数
```

⚠ 误区:哈希与判等缺一不可

只写 `operator==` 不写哈希结构, `unordered_set<Student>` 编译失败;只写哈希不写判等,报错变成"没有匹配的 `operator==`". 标准:相等对象散列值必相等,不等对象尽量分散(异或组合对简单结构足够)。

5.6.3 选型:unordered_map 还是 map

`map` 底层是红黑树,查找 $O(\log n)$ 但按键有序;`unordered_map` 平均 $O(1)$ 但无序。选型口诀:

需求	选型	原因
只要按键快速查值,不关心顺序	unordered_map	平均 $O(1)$,快一个数量级
需要按键有序遍历、范围查询	map	有序,且最坏 $O(\log n)$ 有保证
自定义类型做键	map(省事)或 unordered_map + 哈希	map 只要 <code>operator<</code> ,unordered_map 要两个

实例:按时间戳顺序输出 10 万条日志 → map(有序遍历);按用户 ID 秒查余额 → unordered_map(只求快)。两者共存很常见。

§5.7 手写与标准库并排:单词统计

5.7.1 需求:统计文章词频

统计一篇文章里每个单词的出现次数("the 出现 1800 次"之类)。这是散列表的教科书级应用,也是章末实验的前身。同一需求,两条路线各写一遍,并排对比。

5.7.2 手写版实现

示例 5-10 手写版:桶内存 (单词, 次数) 对

```

// 手写版:桶内放 (单词, 次数),统计每个单词的出现次数
class WordCounter {
    vector<list<pair<string, int>>> buckets_; // 桶数组
    int distinct_ = 0; // 不同单词个数
public:
    explicit WordCounter(int m) : buckets_(nextPrime(m)) {}

    void add(const string& w) { // 出现一次就计数一次
        auto& b = buckets_[hashStr(w, int(buckets_.size()))];
        for (auto& kv : b)
            if (kv.first == w) { ++kv.second; return; } // 已存在:+1
        b.push_back({w, 1}); // 新词:插到链表尾
        ++distinct_;
    }

    int count(const string& w) const { // 查询词频,不存在返回 0
        auto& b = buckets_[hashStr(w, int(buckets_.size()))];
        for (auto& kv : b)
            if (kv.first == w) return kv.second;
        return 0;
    }

    int distinct() const { return distinct_; }
};

int main() {
    WordCounter wc(16); // 表大小 16 → 素数 17
    string text = "the cat and the dog and the cat";
    string cur;
    for (char c : text) { // 切词:非字母数字当分隔
        if (isalnum((unsigned char)c))
            cur += (char)tolower((unsigned char)c);
        else if (!cur.empty()) { wc.add(cur); cur.clear(); }
    }
    if (!cur.empty()) wc.add(cur);
    cout << "the: " << wc.count("the") << "\n"; // 3
    cout << "cat: " << wc.count("cat") << "\n"; // 2
    cout << "不同单词: " << wc.distinct() << "\n"; // 4
    return 0;
}

```

5.7.3 标准库版实现

示例 5-11 标准库版:同一需求,代码量减到 1/3

```
// 标准库版:unordered_map 一行完成"不存在则插入"
#include <unordered_map>
int main() {
    string text = "the cat and the dog and the cat";
    unordered_map<string, int> cnt;
    for (char c : text) { // 切词与手写版一致
        if (isalnum((unsigned char)c)) cur += (char)tolower((unsigned char)c);
        else if (!cur.empty()) { ++cnt[cur]; cur.clear(); }
    }
    if (!cur.empty()) ++cnt[cur];
    cout << "the: " << cnt["the"] << "\n"; // 3
    cout << "cat: " << cnt["cat"] << "\n"; // 2
    cout << "不同单词: " << cnt.size() << "\n"; // 4
    return 0;
}
```

5.7.4 对照结论

C 的写法	C++ 的写法
结构体数组存 (word, cnt), 每次查找线性扫描 $O(n)$	<code>unordered_map<string, int>: ++cnt[w]</code> , 平均 $O(1)$
要自己写字符串比较、手动扩容、手动释放	string 自带 <code>==</code> , 自动再散列, RAII 自动释放



调试技巧: 手写版与库版互相验算

手写数据结构最常见的 bug 是“错得整齐”: 两处实现犯同一个错, 测试却全绿。对策是章末实验的做法——同一份数据喂给手写表和 `unordered_map`, 抽查若干键的词频必须一致。这个“双实现互验”的习惯, 写红黑树、堆时都用得上。

本章复杂度速查表

口诀: 查找/插入/删除平均 $O(1)$, 最坏 $O(n)$; 代价是放弃按键顺序。以下结论两版通用。

操作	数据结构/算法	平均	最坏
查找	散列表(分离链接)	$O(1)$	$O(n)$
插入	散列表(分离链接)	$O(1)$	$O(n)$
删除	散列表(分离链接)	$O(1)$	$O(n)$
查找	散列表(开放定址)	$O(1)$	$O(n)$
插入	散列表(开放定址)	$O(1)$	$O(n)$
删除(惰性)	散列表(开放定址)	$O(1)$	$O(n)$
散列函数	整数取模	$O(1)$	—
散列函数	字符串 37 倍多项式(键长 L)	$O(L)$	—
再散列	单次再散列(翻倍重插)	$O(n)$	$O(n)$
连续插入	含再散列的 n 次插入	摊还 $O(1)$ /次	$O(n)$ /次
查找/插入/删除	unordered_map / unordered_set	$O(1)$	$O(n)$
查找/插入/删除	map / set(红黑树)	$O(\log n)$	$O(\log n)$
按键有序遍历	map / set	$O(n)$	$O(n)$
探测次数	线性/平方探测, $\lambda \approx 0.5$	≈ 1.5 次	$O(n)$
词频统计	手写散列表 / unordered_map	$O(n)$	$O(n^2)$

最小必会清单

- 能说出散列解决什么问题:键映射为下标,查找压到平均 $O(1)$,代价是放弃顺序性。
- 能手写整数取模散列,并处理负数取模的 C++ 截断语义(修正到非负)。
- 能手写 37 倍多项式字符串散列,并解释"为什么先累加、最后取模"。
- 能说出表大小必须取素数,并给出合数表大小的反例。
- 能说出装填因子 $\lambda = N/M$ 的含义与经验阈值(分离链接 ≈ 0.75 ,平方探测 < 0.5)。
- 能实现分离链接散列表的 find / insert / remove,能指出 erase 后迭代器失效的坑。
- 能说出开放定址三种方案的探测序列、聚集现象与适用条件。
- 能解释惰性删除的必要性(物理删除会切断探测链)。
- 能实现再散列,说出"每次插入摊还 $O(1)$ "与 vector 扩容的同构性。
- 能用 unordered_map / unordered_set 解决词频统计等问题,注意 operator[] 的自动插入副作用。
- 能给自定义类型写哈希与 operator==(两个模板参数),并在 unordered_map 与 map 之间选型。

自测题

Q 1. 表大小 $M = 7$,散列函数 $h(x) = x \bmod 7$,用分离链接法依次插入 13、27、41、55。画出各桶的链表,并计算装填因子 λ 与平均链长。

✓ 参考答案

13、27、41、55 除以 7 的余数全是 6——四键全落 6 号桶(最坏情况的构造方式)。 $\lambda = 4/7 \approx 0.57$,平均链长 4/7,但 6 号桶链长 4。

Q 3. 逐字符取模与最后取模,在 $M = 37$ 和 $M = 36$ 时各会怎样退化?

✓ 参考答案

$M = 37$: $(h*37+c) \bmod 37 = c \bmod 37$,退化为"只看最后一个字符"; $M = 36$: $37 \equiv 1 \pmod{36}$,每个字符权重都是 1,"abc" 与 "cba" 同值。根治:先累加、最后取一次模。

Q 4. 表大小 $M = 11$,线性探测依次插入 3、14、25、8、19,给出最终表内容与探测过程。

✓ 参考答案

$3 \rightarrow 3$; $14 \rightarrow 3$ 冲突 $\rightarrow 4$; $25 \rightarrow 3$ 冲突 $\rightarrow 4$ 冲突 $\rightarrow 5$; $8 \rightarrow 8$; $19 \rightarrow 8$ 冲突 $\rightarrow 9$ 。最终下标 3、4、5、8、9 依次为 3、14、25、8、19。14、25 挤成连续块,正是"一次聚集"。

Q 5. 开放定址表里删除元素为什么不能直接置空?工程上怎么处理?

✓ 参考答案

直接置空会把探测链拦腰截断:后续键靠被删元素"借道",探测序列在此断掉,再也找不到。工程做法是惰性删除:标记 deleted,查找跳过、插入复用、定期再散列清标记。

Q 6. 统计 100 万条记录:需求 A 按 key 升序输出,需求 B 只按 key 快速查值。各选 map 还是 unordered_map? 为什么?

✓ 参考答案

需求 A 用 map(红黑树,按键有序遍历 $O(n)$);需求 B 用 unordered_map(平均 $O(1)$ 查找)。口诀:要顺序选树,只要速度选散列。

章末实验题:散列表单词统计对比实验

实验 5:散列表单词统计对比实验

实验目标:手写分离链接散列表,与 `unordered_map` 双路线统计英文文本词频,用计时数据验证装填因子 λ 对性能的影响。

实验要求:

- 任务 1:实现分离链接散列表(键为字符串):37 倍多项式散列、素数表大小、add/count 操作(覆盖 §5.1 散列函数与素数表、§5.2 分离链接法)。
- 任务 2:用自写的表统计文本,输出单词总数、不同单词个数、装填因子 λ (覆盖 §5.1.4 装填因子)。
- 任务 3:用 `unordered_map` 重做统计,抽查 "the"、"hash"、"hello" 三个词,两套结果必须一致(覆盖 §5.6 标准库;兼作手写版正确性验证)。
- 任务 4:重复插入 30 遍,改变表大小 $M = 8/16/64/256$ (实际素数 $11/17/67/257$),每种测 3 次取最快;观察 λ 越大耗时越高,体会再散列的动机(覆盖 §5.2.3 复杂度、§5.4 再散列)。
- 任务 5:健壮性——无命令行参数、文件打不开、文本为空都要给出友好提示(覆盖边界处理与工程习惯)。

实验环境:C++17。编译:`g++ -std=c++17 lab5.cpp -o lab5`;运行:`./lab5 sample.txt`(任意英文文本,词越多对比越明显)。

第一步 写工具函数:`isPrime/nextPrime` 定素数表大小;`splitWords` 切词(字母数字连续成词,转小写,其余字符当分隔)。先用 3-5 个单词手算验证散列。

第二步 写 `HashTable`:构造函数用 `nextPrime` 定桶数;hash 用"先累加、最后取模";add 找到键计数 +1,否则插入新键。写完后用小文本对照字典检查词频。

第三步 后写计时:任务 4 对每个 M 重复 3 次取最快,插入重复 30 遍放大差异;跑前先确认 λ 合理(如 208 个不同单词、 $M=257$ 时 $\lambda \approx 0.81$),再做比较。

✅ 参考答案(完整可运行程序,分三段)

实验 5-1 参考答案(第一部分:工具函数与切词)

```
// lab5.cpp — 散列表单词统计对比实验(第一部分:工具函数)
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <list>
#include <unordered_map>
#include <chrono>
#include <cctype>
using namespace std;
using Clock = chrono::steady_clock;

// ----- 工具:素数判断与下一个素数 -----
bool isPrime(int n) {
    if (n < 2) return false;
    for (int d = 2; d * d <= n; ++d)
        if (n % d == 0) return false;
    return true;
}
int nextPrime(int n) {
    while (!isPrime(n)) ++n;
    return n;
}

// 任务 5:切分单词—字母/数字连续成词并转小写,其余字符跳过
vector<string> splitWords(const string& text) {
    vector<string> words;
    string cur;
    for (char c : text) {
        if (isalnum((unsigned char)c)) {
            cur += (char)tolower((unsigned char)c);
        } else if (!cur.empty()) {
            words.push_back(cur);
            cur.clear();
        }
    }
    if (!cur.empty()) words.push_back(cur);
    return words;
}
```

实验 5-1 参考答案(第二部分:手写分离链接散列表)

```
// ----- 任务 1:分离链接散列表(字符串键,37 倍散列,素数表) -----
class HashTable {
    struct Entry { string key; int cnt; };
    vector<list<Entry>> buckets_; // 桶数组:vector<list<...>>
    int size_ = 0; // 不同单词个数
public:
    explicit HashTable(int m) : buckets_(nextPrime(m)) {}

    int hash(const string& s) const {
        unsigned long long h = 0; // 全程不取模,最后再 % 表大小(见 §5.1.3)
        for (char c : s)
            h = h * 37 + (unsigned char)c;
        return int(h % buckets_.size());
    }

    void add(const string& s) { // 计数 +1,首次出现则插入
        int b = hash(s);
        for (auto& e : buckets_[b])
            if (e.key == s) { ++e.cnt; return; }
        buckets_[b].push_back({s, 1});
        ++size_;
    }

    int count(const string& s) const { // 查询词频,不存在返回 0
        int b = hash(s);
        for (const auto& e : buckets_[b])
            if (e.key == s) return e.cnt;
        return 0;
    }

    int distinct() const { return size_; }
    int bucketCount() const { return int(buckets_.size()); }
    double loadFactor() const { // 装填因子  $\lambda$  = 元素数/表大小
        return double(size_) / buckets_.size();
    }
};
```

实验 5-1 参考答案(第三部分:计时、main 与四种表大小对比)

```
// ----- 计时工具:插入 repeat 遍,放大差异 -----
double msInsert(HashTable& ht, const vector<string>& words, int repeat) {
    auto t0 = Clock::now();
    for (int r = 0; r < repeat; ++r)
        for (const string& w : words) ht.add(w);
    return chrono::duration<double, milli>(Clock::now() - t0).count();
}

int main(int argc, char* argv[]) {
    if (argc < 2) { cout << "用法: lab5 <文本文件>\n"; return 1; }
    ifstream fin(argv[1]);
    if (!fin) { cout << "错误: 无法打开文件 " << argv[1] << "\n"; return 1; }
    stringstream ss;
    ss << fin.rdbuf();
    vector<string> words = splitWords(ss.str());
    if (words.empty()) { // 任务 5:空文本
        cout << "警告: 文本为空或没有合法单词\n";
        return 0;
    }
    cout << "单词总数(含重复): " << words.size() << "\n";

    // 任务 2:手写表统计
    HashTable ht(256);
    msInsert(ht, words, 1);
    cout << "手写表: 不同单词 " << ht.distinct()
        << " 个,装填因子  $\lambda$ =" << ht.loadFactor() << "\n";

    // 任务 3:标准库重做,抽查输出一致性
    unordered_map<string, int> mp;
    for (const string& w : words) ++mp[w];
    cout << "unordered_map: 不同单词 " << mp.size() << " 个\n";
    for (const char* w : {"the", "hash", "hello"})
        cout << " 词 \"" << w << "\": 手写表=" << ht.count(w)
            << " map=" << mp[w] << "\n";

    // 任务 4:不同  $\lambda$  下的插入耗时(表大小不同  $\rightarrow \lambda$  不同,3 次取最快)
    cout << "任务4: 表大小(素数)不同  $\rightarrow \lambda$  不同  $\rightarrow$  插入耗时(3 次取最快):\n";
    for (int m : {8, 16, 64, 256}) {
        double best = 1e300;
        for (int trial = 0; trial < 3; ++trial) { // 重复 3 次取最快
            HashTable t(m);
            double ms = msInsert(t, words, 30);
            if (ms < best) best = ms;
        }
        HashTable t(m);
        msInsert(t, words, 1);
        cout << " M=" << t.bucketCount() << "  $\lambda$ =" << t.loadFactor()
            << " 耗时 " << best << " ms\n";
    }
    cout << "观察:  $\lambda$  越大平均链表越长,耗时越高——这就是再散列(§5.4)的动机.\n";
    return 0;
}
```

编译运行命令与真实输出(sample.txt 为 7 段文章拼成,17520 词、208 个不同单词):

实验 5-1 输出样例(本机实测,数值随机器与负载略有波动,趋势稳定)

```
$ g++ -std=c++17 -O2 lab5.cpp -o lab5 && ./lab5 sample.txt
```

单词总数(含重复): 17520

手写表: 不同单词 208 个, 装填因子 $\lambda=0.809339$

unordered_map: 不同单词 208 个

词 "the": 手写表=1800 map=1800

词 "hash": 手写表=120 map=120

词 "hello": 手写表=120 map=120

任务4: 表大小(素数)不同 $\rightarrow \lambda$ 不同 $\rightarrow$ 插入耗时(3 次取最快):

M=11 $\lambda=18.9091$ 耗时 5.8237 ms

M=17 $\lambda=12.2353$ 耗时 4.59 ms

M=67 $\lambda=3.10448$ 耗时 3.3488 ms

M=257 $\lambda=0.809339$ 耗时 3.0471 ms

观察: λ 越大平均链表越长, 耗时越高——这就是再散列(\$5.4)的动机。

```
$ ./lab5 empty.txt
```

警告: 文本为空或没有合法单词

设计说明:程序分四层,每层对应一个知识点。散列函数写成"先累加、最后取模"的 37 倍多项式(\$5.1.3),表大小交给 nextPrime(\$5.1.2)——本章最易写错的两处,用真实数据检验了正确性。桶用 `vector<list<Entry>>` 实现分离链接(\$5.2),Entry 携带键与计数,一次遍历完成"查重或累加"。任务 3 的抽查是刻意的工程习惯:手写实现与标准库输出一致,才敢相信手写版正确。任务 4 把 M 从 11 调到 257, λ 从 18.9 降到 0.81,耗时从 5.8 ms 降到 3.0 ms——M=11 时平均每链 1589 个词,查找已接近线性扫描,这是" λ 过高 $\rightarrow$ 性能塌方 $\rightarrow$ 再散列"的实证。计时 3 次取最快 + 重复 30 遍,压掉系统调度噪声。切词规则决定了"单词"的定义,是统计程序必须显式声明的约定。

下一章预告:第6章 优先队列(堆)——基于数组的完全二叉树;建堆与删除最小将使用第 3 章的 `vector/list`,堆的插入(上滤)与删除最小(下滤)是第 7 章堆排序、第 9 章 Dijkstra 最短路径的基石。